

Biophysics 21: Biological Light Microscopy

Section 10: Image analysis using Python

“Discussion Section”: Setting up an environment on your computer

Discussion section: Tuesday 1 – 2.30 pm GH N114

1. **Command line interface (CLI).** Identify or install your command line interface. On the Mac this is called “Terminal”. Figure out how to launch Terminal easily. On Windows, there are multiple options. The old fashioned “cmd” DOS prompt is a bit hard to work with, so you may want to prefer PowerShell. I use the bash shell that comes installed by git (see below). Each CLI talks its own “dialect”, but they are all quite similar (except for the DOS prompt).
2. **Git:** download and install git from <https://git-scm.com/install/>. On Windows, this also will install the bash shell. Although not essential, you may want an account on github (<https://github.com>). Note that you can join Github education, which may give some useful benefits such as free private repositories.
3. **Python:** Python is so powerful because there is a very large number of packages that provide functionality you will want to use. For instance, almost everything you do will include the ‘numpy’ package that provides support for numerical methods including matrix calculations. ‘scikit-image’ is a foundational package for image analysis. ‘napari’ or ‘ndv’ can be used for visualization of multi-dimensional images. These packages all depend on each other. Problems can arise when versions change and these dependencies no longer work together (also called “dependency hell”). To mitigate these problems, we like to work in a “virtual environment”, so that the dependencies you use for one project do not interfere with the dependencies of another project and nuke your only python environment. This can be managed with the ‘venv’ tool built-in python or using the popular ‘conda’ tool. Recently, a new tool, called “uv” has become popular due to its speed, reliability and efficiency. Here, we will be using uv. To install uv follow this link: <https://docs.astral.sh/uv/getting-started/installation/>.
4. **Generative AI:** You can probably get far by using a web interface to AI such as ChatGTP, however, much can be gained by giving an AI access to the files in your project. I very much like “Claude Code” for this purpose, however, Claude Code needs a subscription with Anthropic (\$20 per month, \$200 per year), whereas Codex currently is free. Please note that UCSF Enterprise ChatGPT does not make Codex

available (because of HIPA concerns), so you may need to use a private email to sign up with ChatGPT to install codex (<https://developers.openai.com/codex/cli>).

5. Putting it together: Clone a github repository and install a python environment for it:

- On the command line interface go to a directory where your project will be located. For instance (commands to type into the CLI are in bold face:

mkdir ~/projects

cd ~/projects

This will create a directory “projects” in your home directory and you will move to that directory.

Optional: Fork the nicost/cellquant repository on github and clone that.

Most everyone else should:

git clone https://github.com/nicost/cellquant.git

cd cellquant

git checkout BP210

You cloned the cellquant repository with code and example images from github and have a copy on your computer. You checked out the BP210 branch of the repo.

Now create a virtual environment and download all dependencies as follows:

uv sync

This creates a virtual environment in the directory “.venv”. Run commands as follows:

uv run cellquant.py

This will launch python, and execute the code in the file cellquant.py

If you got here: Celebrate!